

TRABAJO DE GRADO · 2026 · UNITY 6 LTS

ONE PUNCH DELUXE

Documentación técnica completa — videojuego de boxeo 2D arcade

- 🥊 Vista lateral KOF
- 🎭 Modo historia
- 🤖 IA con FSM
- 🎮 Soporte mando Xbox
- 🐍 Pipeline Python (Pillow + rembg)
- 🎵 Sistema de audio completo

UNITY 6	C#	19	18	15+
MOTOR	LENGUAJE	SPRITES	ANIMACIONES	SCRIPTS PYTHON

Ronald Medina · Ing. de Sistemas

github.com/royemequi-byte/one-punch-deluxe

01 EL JUEGO

ONE PUNCH DELUXE es un videojuego de boxeo 2D arcade estilo **KOF (King of Fighters)** — vista lateral con los dos personajes enfrentados — desarrollado en Unity 6 como trabajo de grado universitario. El jugador controla a **Wally**, un boxeador con historia propia, contra el rival **Máximo / Kirikú** en un ring profesional con multitud animada.

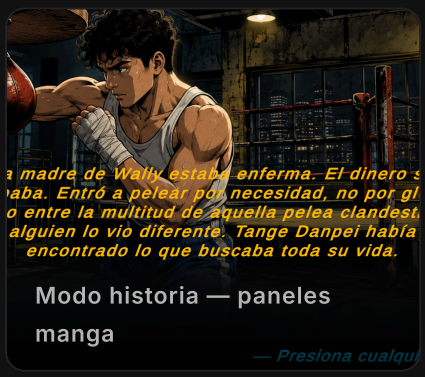
El juego cuenta con modo historia completo (paneles de arte manga con narrativa), sistema de combate con tres golpes + esquives + bloqueo, HUD con barra de vida y stamina, sistema de rounds, pantalla de game over, guardado en JSON y soporte para mando Xbox.



Combate — Ring con multitud, HUD completo, timer



Menú principal



Modo historia — paneles manga



Wally vs Máximo — gancho



Pantalla Game Over

02 STACK TECNOLÓGICO

CATEGORÍA	TECNOLOGÍA	USO ESPECÍFICO EN EL PROYECTO
MOTOR	**Unity 6 LTS · URP 2D**	Universal Render Pipeline, modo 2D ortográfico
LENGUAJE	**C#**	Scripts MonoBehaviour, ScriptableObjects, FSM de IA
INPUT	**Unity Input System**	Teclado + mando Xbox / PlayStation
UI	**TextMeshPro + Canvas**	HUD, menús, pantalla de game over
DATOS	**ScriptableObjects + JsonUtility**	Perfiles de IA, datos de ataque, guardado
ARTE	**Adobe Illustrator**	Diseño de interfaz: HUD, botones, barras de vida, elementos UI y composición visual del juego
ARTE	**Python + Pillow + rembg**	Pipeline completo de preparación de sprites
ARTE	**IA generativa**	Fondos (ring, multitud), ilustraciones de historia
AUDIO	**AudioMixer nativo Unity**	Música, SFX de golpes, efectos de ambiente
GIT	**Git + GitHub + Git LFS**	Control de versiones; LFS para PNG y WAV

03 ARQUITECTURA DE LA ESCENA

La escena principal es `Assets/dia 1.unity`. Organización por SortingOrder:

GAMEOBJECT	POSICIÓN	ESCALA	LAYER	NOTAS
BackgroundRing	`(0, 0, 0)`	`(0.926, 0.926, 1)`	-3	ring_bg.png 1920×1080 · generado con Python
Kiriku	`(3, -1.5, 0)`	`(-3, 3, 1)`	1	Rival — flipX por escala X negativa
Wally	`(-3, -1.5, 0)`	`(2.5, 2.5, 1)`	2	Jugador — lado izquierdo
Canvas / HUD	—	—	Overlay	HUDController: barras de vida, stamina, timer, controles

> **Flipeo de Kiriku:** la escala X negativa (`-3`) voltea el sprite automáticamente para que mire a la izquierda — sin necesidad de sprites espejo ni código adicional.

Pantallas implementadas

PANTALLA	DESCRIPCIÓN
SPLASH	Pantalla de inicio con logo
MENÚ PRINCIPAL	Ring desde arriba con opciones NUEVA PARTIDA / CONTINUAR / SALIR
HISTORIA	Paneles de arte manga con texto narrativo de la historia de Wally
COMBATE	Ring lateral KOF con HUD completo, timer, barras y controles
GAME OVER	Banner rojo, marcador de rounds, botones REINTENTAR / MENÚ PRINCIPAL
RESULTADO	Pantalla de victoria con estadísticas

04 SPRITES DE WALLY

Ruta: `Assets/Art/Personajes/Wally_Nuevos/` · Pivot: **Bottom Center (0.5, 0)** · Pies en Y = -1.5 · Cabeza ≈ Y +3.375 con escala 2.5

> Los sprites de `Assets/Art/Personajes/Wally/` (vista de espalda, estilo Punch-Out) están **descartados**. No usar.



Idle

`wally_idle.png`

Guardia en reposo. Puños en alto.

WALLY_IDLE · 5 FRAMES · LOOP



Ataque

`wally_straight.png`

Brazo extendido. Cubre Jab, Straight y Hook.

JAB · STRAIGHT · HOOK



Bloqueo

`wally_block.png`

Antebrazos cruzados frente al rostro.

BLOCK · WEAVELEFT



Agachado

`wally_block2.png`

Postura baja. Esquive y retroceso.

DUCK · BACKSTEP · WEAVERIGHT



Recibir golpe

`wally_hit.png`

Cabeza hacia atrás por impacto.

WALLY_HIT.ANIM

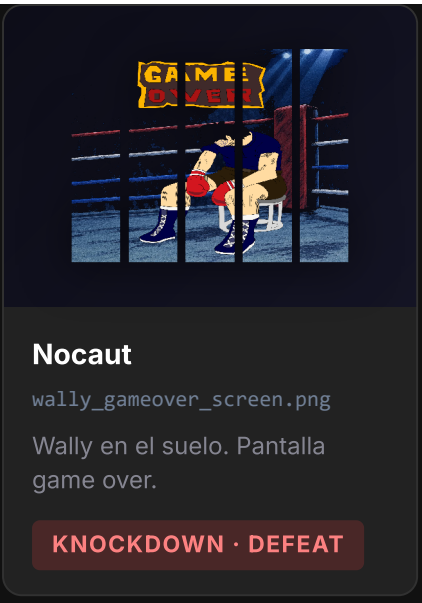


Golpe fuerte

`wally_hit2.png`

Reacción exagerada a impacto de alto daño.

HIT ALTERNATIVO



Nocaut

`wally_gameover_screen.png`

Wally en el suelo. Pantalla game over.

KNOCKDOWN · DEFEAT

05 SPRITES DE KIRIKÚ

Ruta: `Assets/Art/Personajes/Kiriku/` · Posición X = +3 · Escala `(-3, 3, 1)` · Canvas de normalización: 1370 × 353 px



Idle

`kiriku_idle.png`

Guardia base. Frame 1 del loop.

KIRIKU_IDLE.ANIM



Idle 2

`kiriku_idle2.png`

Variación de respiro. Frame 2.

KIRIKU_IDLE.ANIM



Idle 3

`kiriku_idle3.png`

Tercer frame del loop de idle.

KIRIKU_IDLE.ANIM



Jab

`kiriku_jab.png`

Jab rápido. Bajo daño, alta velocidad.

KIRIKU_JAB.ANIM



Straight

`kiriku_attack.png`

Golpe recto extendido. Daño medio.

KIRIKU_STRAIGHT.ANIM



Hook

`kiriku_hook.png`

Gancho lateral. Mayor daño, más lento.

KIRIKU_HOOK.ANIM

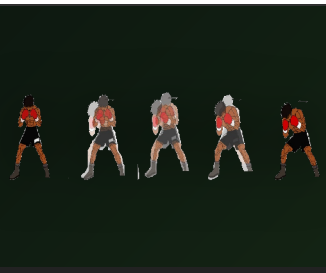


Bloqueo

`kiriku_block.png`

Brazos en cruz. Reduce daño recibido.

KIRIKU_BLOCK.ANIM



Esquive

`kiriku_weave.png`

Cuerpo inclinado para esquivar.

KIRIKU_WEAVE.ANIM



Telegrafiar

`kiriku_telegraph.png`

Aviso visible antes de ataque fuerte. Exclusivo de la IA.

TELEGRAPH · SOLO IA

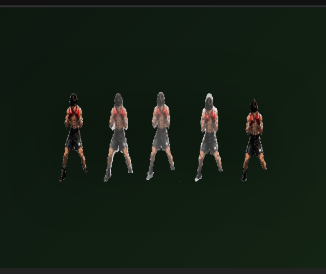


Recibir golpe

`kiriku_hit.png`

Cabeza hacia atrás por impacto.

KIRIKU_HIT.ANIM



Golpe fuerte

`kiriku_hit2.png`

Reacción exagerada. Mayor impacto.

HIT VARIANTE



Nocaut

`kiriku_knockdown.png`

Kirikú en el suelo. Victoria del jugador.

KNOCKDOWN · DEFEAT

06 SISTEMA DE COMBATE

J	**K**	**L**	**SP**
Jab	**Straight**	**Hook**	**Bloqueo**
Rápido · bajo daño	Medio · equilibrado	Lento · alto daño	Space · reduce daño
W	**A**	**S**	**D**
WeaveLeft	**Backstep**	**Duck**	**WeaveRight**
Esquive izq.	Retroceder	Agacharse	Esquive der.

> Los esquives son **posturas fijas**, no movimiento libre. También funciona con mando Xbox / PlayStation vía Unity Input System.

ScriptableObjects de ataque — `Assets/ScriptableObjects/Attacks/`

ASSET	ATAQUE	CONTENIDO
`JabData.asset`	Jab	Daño, velocidad, frames activos, hitbox
`StraightData.asset`	Straight	Daño, velocidad, frames activos, hitbox
`HookData.asset`	Hook	Daño, velocidad, frames activos, hitbox

IA de Kirikú — FSM de 7 estados

`RivalAI.cs` carga `KirikuProfile.asset` y ejecuta una máquina de estados: **Idle** → **Telegraph** → **Attack** → **Recovery** → **Idle**, con bifurcaciones a Block, Dodge y Stunned. El estado **Telegraph** (0.25s) es visible para el jugador — la animación `kiriku_telegraph.png` aparece antes de cada golpe fuerte, dando una ventana de esquive.

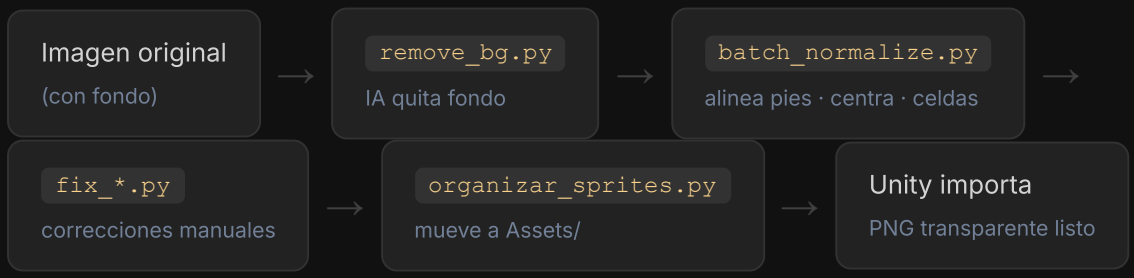
07 PYTHON — PIPELINE DE ARTE

Python se usó exclusivamente como herramienta de producción de assets en `Tools/`. Regla del proyecto: *"C# ejecuta el juego, Python prepara lo que C# consume."*

Librerías — `Tools/requirements.txt`

LIBRERÍA	VERSIÓN	PARA QUÉ SE USÓ
Pillow	`≥10.0.0`	Manipulación de imágenes: recorte, redimensionado, composición, gradientes, canal alpha
NumPy	`≥1.24.0`	Operaciones sobre arrays de píxeles — detección de contenido, alineación de pies entre frames
rembg	`≥2.0.0`	Eliminación de fondo con IA (modelo U²-Net) — hace los sprites transparentes

Flujo completo de un sprite



Scripts implementados

`remove_bg.py`

Usa rembg (IA) para eliminar el fondo. Guarda `_transparent.png` junto al original.

`batch_normalize.py`

Separa frames, alinea pies al mismo nivel base, centra en celda uniforme y exporta sprite sheet listo para Unity.

`normalizar_kiriku.py`

Detecta filas con contenido via canal alpha, recorta y escala a 353px de alto. No toca sprites que ya tienen ≥98% del alto.

`generar_ring.py`

Genera el ring 1920×1080 programáticamente: gradiente oscuro de arena, 350 siluetas de multitud, focos y cuerdas.

make_transparent.py

Convierte fondo de color sólido a transparente (alpha=0) para sprites que no pasaron por rembg.

procesar_wally.py / procesar_kiriku.py

Pipeline completo por personaje: remove_bg → normalizar → corregir → exportar a su carpeta en `Assets/`.

fix_wally_ghosts.py

Elimina píxeles fantasma (artefactos semitransparentes) que dejaba rembg en los bordes del sprite.

organizar_sprites.py

Mueve y renombra sprites procesados a las carpetas correctas de `Assets/Art/Personajes/`.

08 SCRIPTS C# PRINCIPALES

SCRIPT	RESPONSABILIDAD	NOTA CLAVE
`FightManager.cs`	Posiciona personajes al iniciar el combate	Wally (-3,-1.5) · Kiriku (3,-1.5) con escala y flip correctos
`HUDController.cs`	Barras de vida, stamina, timer, controles en pantalla	`BuscarGO()` tolera espacios al inicio del nombre del GO
`RivalAI.cs`	FSM de 7 estados para el comportamiento de Kirikú	Lee `KirikuProfile.asset` — agresividad y patrones configurables
`AudioManager.cs`	Reproduce música y SFX con pool de fuentes	Singleton null-safe con `?.` · usa AudioMixer con dB correcto
`SaveManager.cs`	Guarda y carga progreso con `JsonUtility`	Singleton null-safe
`GameManager.cs`	Flujo general: escenas, estados, rounds	Singleton null-safe · gestiona transición entre pantallas
`VisualJuice.cs`	Efectos de cámara y ring	`CrearFondoRing()` fija escala a 0.926 · pendiente de integrar en escena

Patrón null-safe: todos los Managers usan `?.` para que su ausencia en escena no genere `NullReferenceException`. El juego no crashea si un sistema está en desarrollo.

09 PIPELINE GIT

ASPECTO	CONFIGURACIÓN
Repositorio	`github.com/royemequi-byte/one-punch-deluxe`
Rama activa	`main` (rama única)
Estado oct 2026	Limpio — sin cambios pendientes
Assets grandes	Git LFS — `*.png`, `*.wav` y binarios Unity

> **¿Por qué Git LFS?** Los PNG y WAV de Unity pueden superar el límite de 100 MB de GitHub. Git LFS reemplaza esos archivos por punteros ligeros y almacena el contenido real aparte, manteniendo el historial ágil.

Convención de commits

PREFIJO	CUÁNDO SE USA
`feat:`	Nueva funcionalidad
`fix:`	Corrección de bug
`audio:`	Cambios en AudioManager o clips
`tools:`	Scripts Python en `Tools/`
`docs:`	Documentación
`refactor:`	Limpieza sin cambio de comportamiento